Data Mining: Frequent Pattern Mining

Fall 2026 (7761)

Dept. of Computer Science and Engineering
Instructor: Ting-Wei Lin

2026/10/02, Week 4

What are Frequent Patterns?

- Frequent patterns include:
  - Frequent itemset: items that often appear together in the same transaction. ($\{Milk, Bread\}$)
  - Sequential pattern: events or purchases that often occur in a particular order. (If You Give a Mouse a Cookie)
  - Structured pattern: frequently occurring substructures, such as subgraphs or subtrees.
- In this lecture, we focus on frequent itemsets and association rules.

Association Rule Mining

- Given a collection of transactions, discover relationships among items.
- A rule has the form:

$$X \rightarrow Y$$

  - X: antecedent
  - Y: consequent
  - X and Y appear in the same transaction
- Interpretation: transactions containing X also tend to contain Y.

$$\{Milk, Bread\} \rightarrow \{Butter\}$$

Association analysis

- A rule has the form:

$$X \rightarrow Y, X \bigcap Y = \emptyset$$

- Association analysis discovers recurring relationships among items or attributes in data.
  - It expresses an observed co-occurrence pattern, not causation.

① What can it discover?

- Associations: items that tend to occur together
  - Support, Confidence
- Correlations: whether the co-occurrence is positive or negative
  - Lift

Association analysis (cont.)

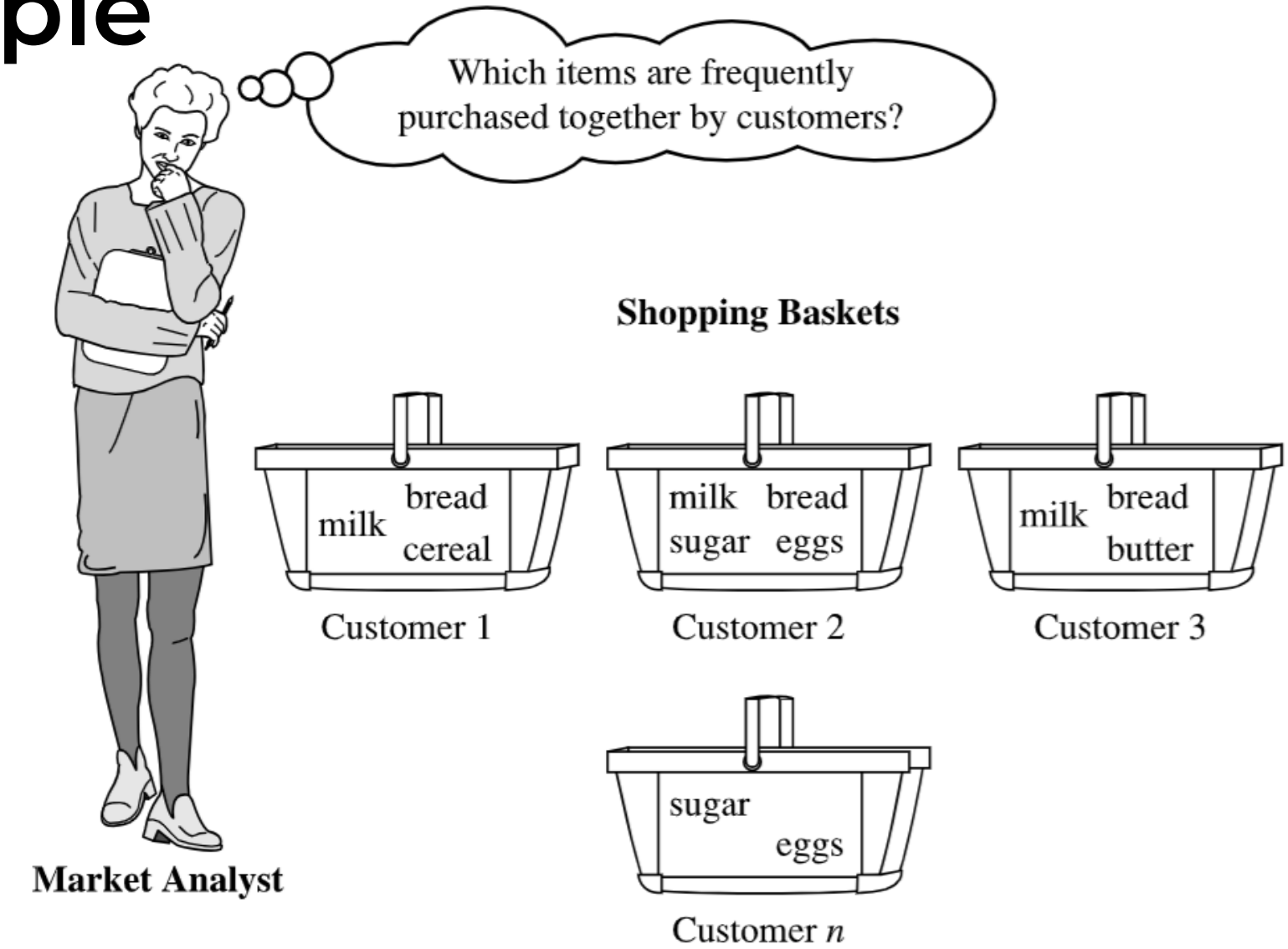
② Where can it be applied?

- Transactional data: items purchased in the same transaction
- Relational data: records with multiple attributes, such as customer profiles and purchases (e.g., $\{Age = 20\text{–}29, Buys = Laptop\}$)

③ How can the discovered patterns be used?

- Market basket analysis
- Product recommendation / cross-selling
- Business decision-making

Market basket analysis: a motivating example



Market basket analysis: a motivating example (cont.)

- Frequent itemset mining discovers recurring co-occurrence patterns in transactional data.
  - Help in many business decision-making processes

① Which products are often purchased together?

② When customers buy milk, what other products are likely to be in the same transaction?

③ Which associations are useful for business decisions?

Market basket analysis: a motivating example (cont.)

- Analyze customer buying habits by finding associations between the different items

$$\{Milk\} \rightarrow \{Bread\}$$
$$\{Milk, Bread\} \rightarrow \{Butter\}$$

- Retailers can use these patterns for product bundles, recommendations, promotions, and shelf placement.

An association rule describes co-occurrence, not causation.

Basic Concepts: Transactional Database

- Transactional Database (TDB)
  - A transactional database contains a set of transactions.
  - Each transaction has a unique identifier, called a transaction ID (TID).
  - Each transaction contains a set of items.
  - In some applications, the quantity of each item may also be recorded.

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

k-Itemsets and Their Supports

- k-itemset: an itemset containing k items:
  - $\{b, c, d\}$ is a 3-itemset
- Support: the count (percentage) of transactions in D that k-itemset
  - Support count (Absolute Support) of an itemset X: the number of transactions in D that contain X.

$$support(\{c, d\}) = 3$$

  - Support (Relative Support) of an itemset X: the fraction of transactions in D that contain X.

$$support(\{c, d\}) = \frac{3}{4} = 75\%$$

Frequent Itemsets (Support)

- An itemset (or a pattern) X is frequent if the support of X is no less than a minimum support threshold σ
  - Let σ = 50% (σ: minsup threshold)
  - Frequent 1-itemset:
    - {a}: 75%, {b}: 50%, {c}: 75%, {d}: 100%
  - Frequent 2-itemset:
    - {a, c}: 50%, {a, d}: 75%, {b, c}: 50%, {b, d}: 50%, {c, d}: 75%
  - Frequent 3-itemset:
    - {b, c, d}: 50%, {a, c, d}: 50%
  - Frequent 4-itemset: None

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Frequent Itemsets (Support Count)

- An itemset (or a pattern) X is frequent if the support count of X is no less than a minimum support threshold σ
  - Let σ = 2 (σ: minsup count threshold)
  - Frequent 1-itemset:
    - {a}: 3, {b}: 2, {c}: 3, {d}: 4
  - Frequent 2-itemset:
    - {a, c}: 2, {a, d}: 3, {b, c}: 2, {b, d}: 2, {c, d}: 3
  - Frequent 3-itemset:
    - {b, c, d}: 2, {a, c, d}: 2
  - Frequent 4-itemset: None

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Definitions of association rules

- $I = \{i_1, i_2, \ldots, i_n\}$, the set of all items
- $T \subseteq I$, a transaction, represented as a nonempty itemset (each transaction is associated with an identifier, TID)
- $D = \{T_1, T_2, \ldots, T_m\}$, a transactional database
- $X \rightarrow Y$, an association rule, where $X \subset I, Y \subset I, X \neq \emptyset, Y \neq \emptyset, X \bigcap Y = \emptyset$
  - $I = \{a, b, c, d, e\}$
    - Is $\{a, c\} \rightarrow \{d\}$ an association rule?

      ✓$\{a, c\} \subset I$, $\{d\} \subset I$

      ✓$\{a, c\} \cap \{d\} = \emptyset$

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

How do we compute the strength of an association rule $X \rightarrow Y$

Both *X* and *Y* are nonempty, disjoint itemsets.



Not every association rule is useful.
We need measures to identify the rules worth keeping.

Support and Confidence

- Support and confidence measure different aspects of a rule.

① Support: the percentage of transactions in D that contain $X \cup Y$

- How common is the rule in the entire database?

$$support(X \to Y) = support(X \cup Y) = \frac{\#\{X \cup Y\}}{|D|}$$

- $support(\{a, d\} \to \{c\}) = support(\{a, c, d\}) = \frac{2}{4} = 50\%$

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Support and Confidence

② Confidence: the percentage of transactions in D containing X that also contain Y

- Among transactions containing X, how often do they also contain Y?

$$confidence(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

- $confidence(\{a, d\} \rightarrow \{c\}) = \frac{support(\{a,c,d\})}{support(\{a,d\})} = \frac{2}{3} \approx 66.7\%$

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Interpreting Support and Confidence

- High support: the rule applies to many transactions.
- High confidence: transactions containing the antecedent often also contain the consequent.
- A strong rule satisfies both minimum support and minimum confidence thresholds.
- Support and confidence alone do not measure correlation or causation.
  - Only show how often data items appear together

Practice



- $\{a\} \rightarrow \{d\}$
  - Support ?
  - Confidence ?
- $\{b, c\} \rightarrow \{e\}$
  - Support ?
  - Confidence ?
- $\{d\} \rightarrow \{a\}$
  - Support ?
  - Confidence ?

$$support(X \rightarrow Y) = support(X \cup Y)$$

$$confidence(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Practice (cont.)

- $\{a\} \rightarrow \{d\}$
  - Support 75%
  - Confidence 100%
- $\{b, c\} \rightarrow \{e\}$
  - Support 25%
  - Confidence 50%
- $\{d\} \rightarrow \{a\}$
  - Support 75%
  - Confidence 75%

$$support(X \rightarrow Y) = support(X \cup Y)$$

$$confidence(X \rightarrow Y) = \frac{support(A \cup B)}{support(A)}$$

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Mining Frequent Itemsets and Association Rules

- Given a set of transactions T
  - The goal of association rule mining is to find all rules satisfying

$$X \rightarrow Y \; is \; a \; strong \; rule \; if$$
$$support(X \cup Y) \geq minsup$$
$$confidence(X \rightarrow Y) \geq minconf$$

  - A strong rule is frequent enough and reliable enough under the chosen thresholds.

Example

- Given
  - Minimum support: 50%
  - Minimum confidence: 66%
- Association rules
  - $\{a\} \rightarrow \{d\},\ \ s = 75\%,\ c = 100\%$
  - $\{d\} \rightarrow \{a\},\ \ s = 75\%,\ c = 75\%$
  - $\{a, d\} \rightarrow \{c\},\ s = 50\%, c = 66\%$
  - $\{a, c\} \rightarrow \{d\},\ s = 50\%,\ c = 100\%$
  - ~~$\{b\} \rightarrow \{c, e\},\ \ s = 25\%,\ c = 50\%$~~

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

The Same Example Using Support Count

- Given
  - Minimum support count: 2
  - Minimum confidence: 66%
- Association rules
  - $\{a\} \rightarrow \{d\},\ \ s = 3,\ c = 100\%$
  - $\{d\} \rightarrow \{a\},\ \ s = 3,\ c = 75\%$
  - $\{a, d\} \rightarrow \{c\},\ \ s = 2,\ c = 100\%$
  - $\{a, c\} \rightarrow \{d\},\ \ s = 2,\ c = 66\%$
  - ~~$\{b\} \rightarrow \{c, e\},\ \ s = 1,\ c = 50\%$~~

TID	Items
00001	a, c, d
00002	b, c, d, e
00003	a, b, c, d
00004	a, d

Association Rule Mining: Basic Concepts

- Itemset: a set of items
- Frequent itemset: an itemset whose support (or count) is at least the minimum support threshold.
- Association rule:$X \rightarrow Y$, where X and Y are nonempty, disjoint itemsets
  - Support: how common $(X \cup Y)$ is in the database
  - Confidence: how often Y occurs when X occurs

How to generate association rules?

- Process of association rule mining
  - Step 1: Find frequent itemsets
  - Step 2: Generate rules from frequent itemsets and retain those satisfying the minimum confidence threshold.
- Why find frequent itemsets first?
- For any association rule

$$X \rightarrow Y$$

its support is the support of the combined itemset:

$$support(X \rightarrow Y) = support(X \cup Y)$$

Why find frequent itemsets first?

- If the rule satisfies the minimum support threshold,

$$support(X \rightarrow Y) = support(X \cup Y) \geq minsup$$
$$\Rightarrow X \cup Y \; is \; a \; frequent \; itemset$$

- A strong rule must come from a frequent itemset.
  - We first find all frequent itemsets.
- How to obtain rules from a frequent itemset?

From frequent itemset to Candidate rules

- For each frequent itemset L, split L into X and $L - X$ to form candidate rules:

$$X \rightarrow L - X$$

- Let $L = \{b, c, d\}$

$\{b\} \rightarrow \{c, d\}$, $\{c\} \rightarrow \{b, d\}$, $\{d\} \rightarrow \{b, c\}$

$\{b, c\} \rightarrow \{d\}$, $\{b, d\} \rightarrow \{c\}$, $\{c, d\} \rightarrow \{b\}$

Why Frequent Itemset Mining Is Difficult

- A low minimum support threshold can produce a huge number of frequent itemsets.
- The output often contains redundant information.
  - If an itemset is frequent, each of its subsets is frequent as well
    - $\{b, c, d\}$ is frequent, all its subset
    - $\{b\}, \{c\}, \{d\}, \{b, c\}, \{c, d\}, \{b, d\}$ are frequent?
- Reduce the number of patterns we report.
  ① Closed frequent itemset
  ② Maximal frequent itemset

① Closed frequent itemset

- A frequent itemset A is closed if there exists no itemset B such that
  a. A is a proper superset of B
  b. Every transaction containing A also contains B
  - $\mathrm{A}\ is\ closed\ if\ there\ is\ no\ frequent\ \mathrm{B} \supset \mathrm{A}\ such\ that\ sup(\mathrm{A}) = sup(\mathrm{B})$
- Closed frequent itemsets preserve exact support information.
  - If A is not closed, there exists a larger itemset $\mathrm{B} \subset \mathrm{A}$ with the same support.
  - We can report B instead of A without losing the support of A.

Closed frequent itemset: Example

min_sup count = 2

TID	Items
00001	a, c, d
00002	b, c, d
00003	a, b, c, d
00004	a, d



Itemset	Support
$\{a\}$	3
$\{b\}$	2
$\{c\}$	3
$\{d\}$	4
$\{a, b\}$	1
$\{a, c\}$	2
$\{a, d\}$	3
$\{b, c\}$	2
$\{b, d\}$	2
$\{c, d\}$	3

Itemset	Support
$\{a, b, c\}$	1
$\{a, b, d\}$	1
$\{a, c, d\}$	2
$\{b, c, d\}$	1
$\{a, b, c, d\}$	1

Is {a} a closed frequent itemset?

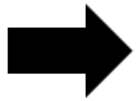
✓ {a} is frequent

- {a}'s super itemset
  - ➢ {a, b}, {a, c}, {a, d}

Closed frequent itemset: Example

min_sup count = 2

TID	Items
00001	a, c, d
00002	b, c, d
00003	a, b, c, d
00004	a, d



Itemset	Support
$\{a\}$	3
$\{b\}$	2
$\{c\}$	3
$\{d\}$	4
$\{a, b\}$	1
$\{a, c\}$	2
$\{a, d\}$	3
$\{b, c\}$	2
$\{b, d\}$	2
$\{c, d\}$	3

Itemset	Support
$\{a, b, c\}$	1
$\{a, b, d\}$	1
$\{a, c, d\}$	2
$\{b, c, d\}$	1
$\{a, b, c, d\}$	1

② Maximal frequent itemset

- A frequent itemset A is maximal if there exists no itemset set B such that
  a. B is a proper superset of A
  b. B is not frequent
  - $X\ is\ maximal\ if\ there\ is\ no\ frequent\ Y \supset X$

Maximal frequent itemset: Example

min_sup count = 2

TID	Items
00001	a, c, d
00002	b, c, d
00003	a, b, c, d
00004	a, d



Itemset	Support
$\{a\}$	3
$\{b\}$	2
$\{c\}$	3
$\{d\}$	4
$\{a, b\}$	1
$\{a, c\}$	2
$\{a, d\}$	3
$\{b, c\}$	2
$\{b, d\}$	2
$\{c, d\}$	3

Itemset	Support
$\{a, b, c\}$	1
$\{a, b, d\}$	1
$\{a, c, d\}$	2
$\{b, c, d\}$	1
$\{a, b, c, d\}$	1

Maximal frequent itemset: Example

min_sup count = 2

TID	Items
00001	a, c, d
00002	b, c, d
00003	a, b, c, d
00004	a, d



Itemset	Support
$\{a\}$	3
$\{b\}$	2
$\{c\}$	3
$\{d\}$	4
$\{a,b\}$	1
$\{a,c\}$	2
$\{a,d\}$	3
$\{b,c\}$	2
$\{b,d\}$	2
$\{c,d\}$	3

Itemset	Support
$\{a,b,c\}$	1
$\{a,b,d\}$	1
$\{a,c,d\}$	2
$\{b,c,d\}$	1
$\{a,b,c,d\}$	1

Total itemset: 15
Frequent: 10
Infrequent: 5

Maximal frequent itemset: Example

min_sup count = 2

TID	Items
00001	a, c, d
00002	b, c, d
00003	a, b, c, d
00004	a, d

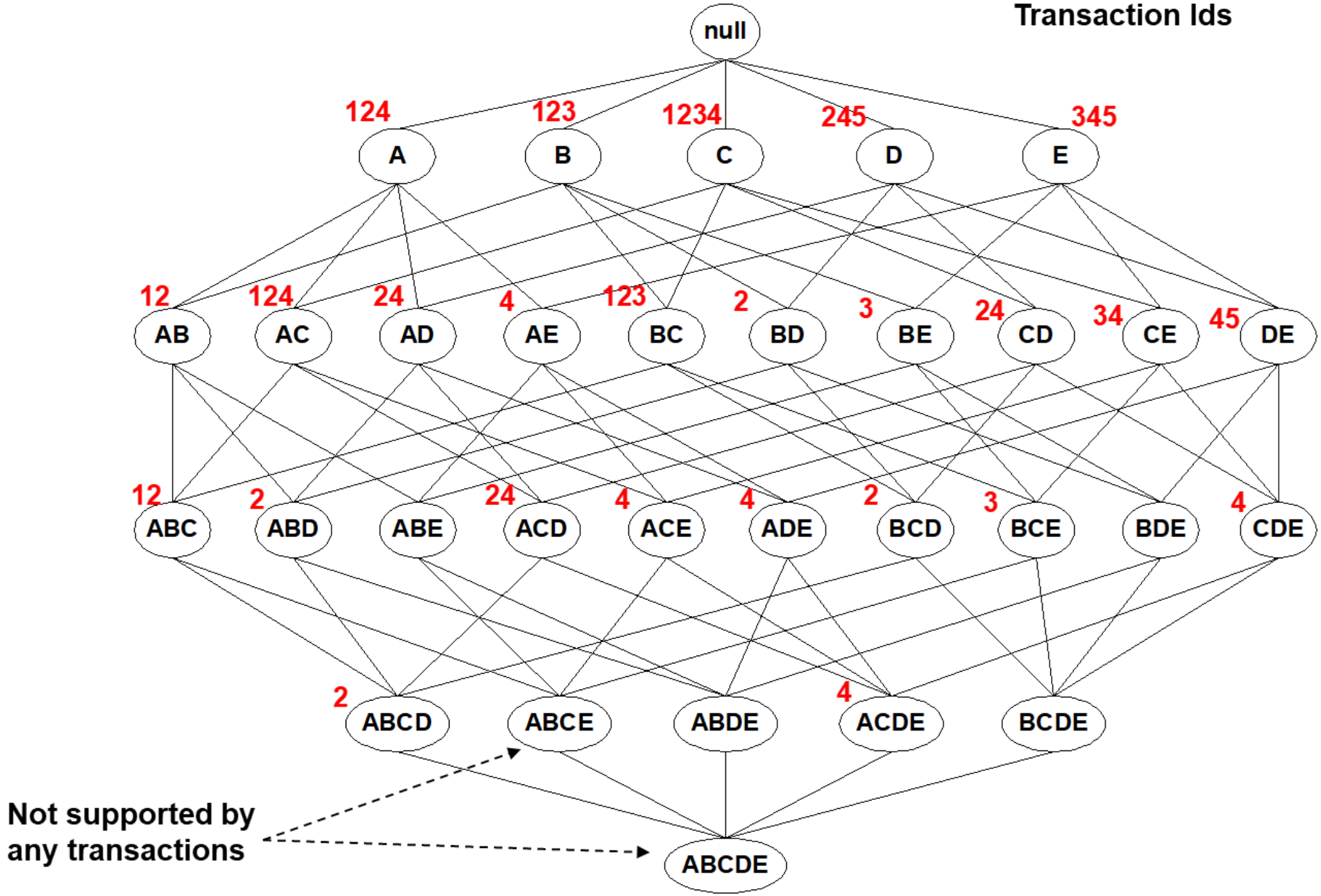


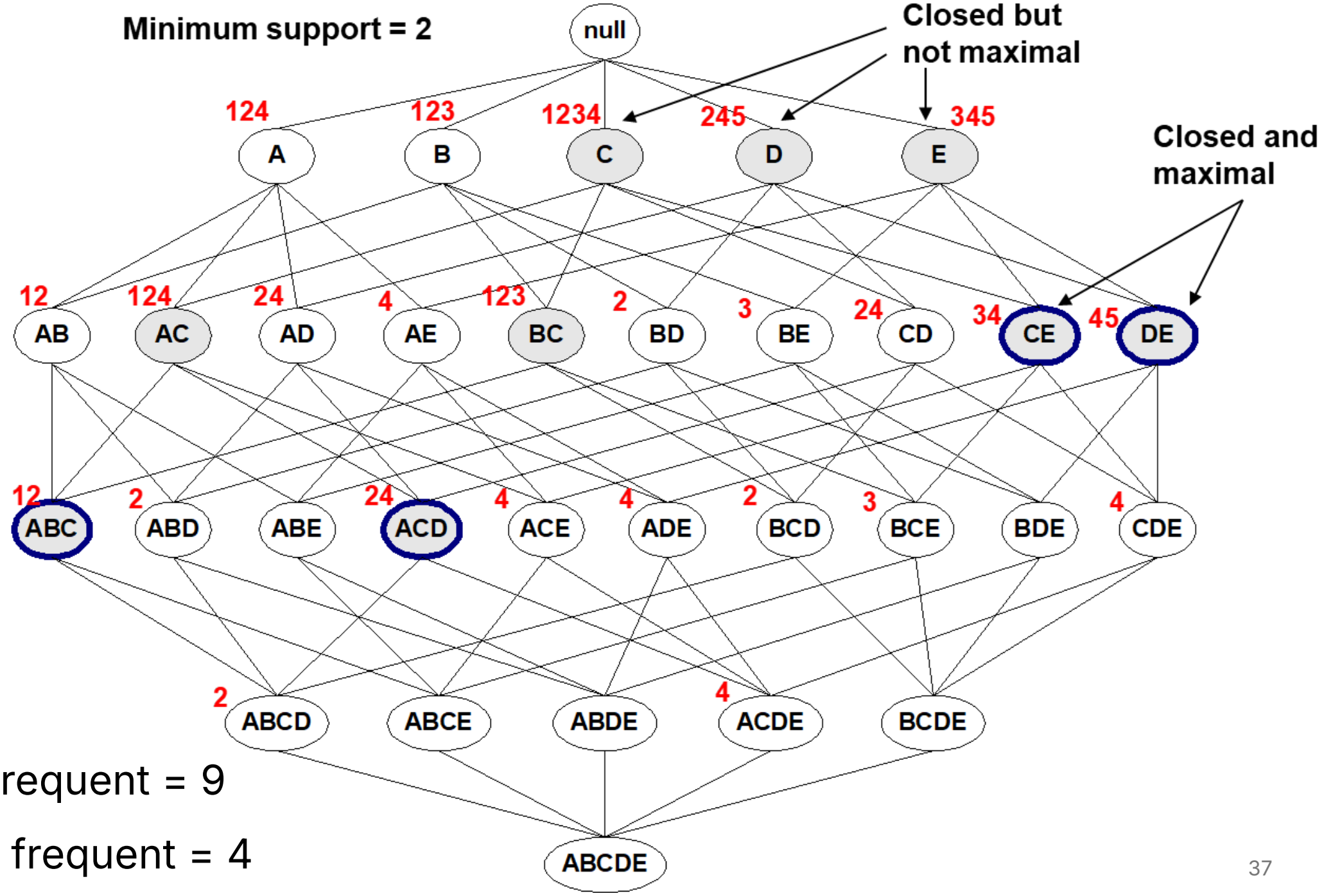
Itemset	Support
$\{a\}$	3
$\{b\}$	2
$\{c\}$	3
$\{d\}$	4
$\{a, b\}$	1
$\{a, c\}$	2
$\{a, d\}$	3
$\{b, c\}$	2
$\{b, d\}$	2
$\{c, d\}$	3

Itemset	Support
$\{a, b, c\}$	1
$\{a, b, d\}$	1
$\{a, c, d\}$	2
$\{b, c, d\}$	1
$\{a, b, c, d\}$	1

Maximal frequent itemset
{b, c}
{b, d}
{a, c, d}

No frequent superset exists for these itemsets.

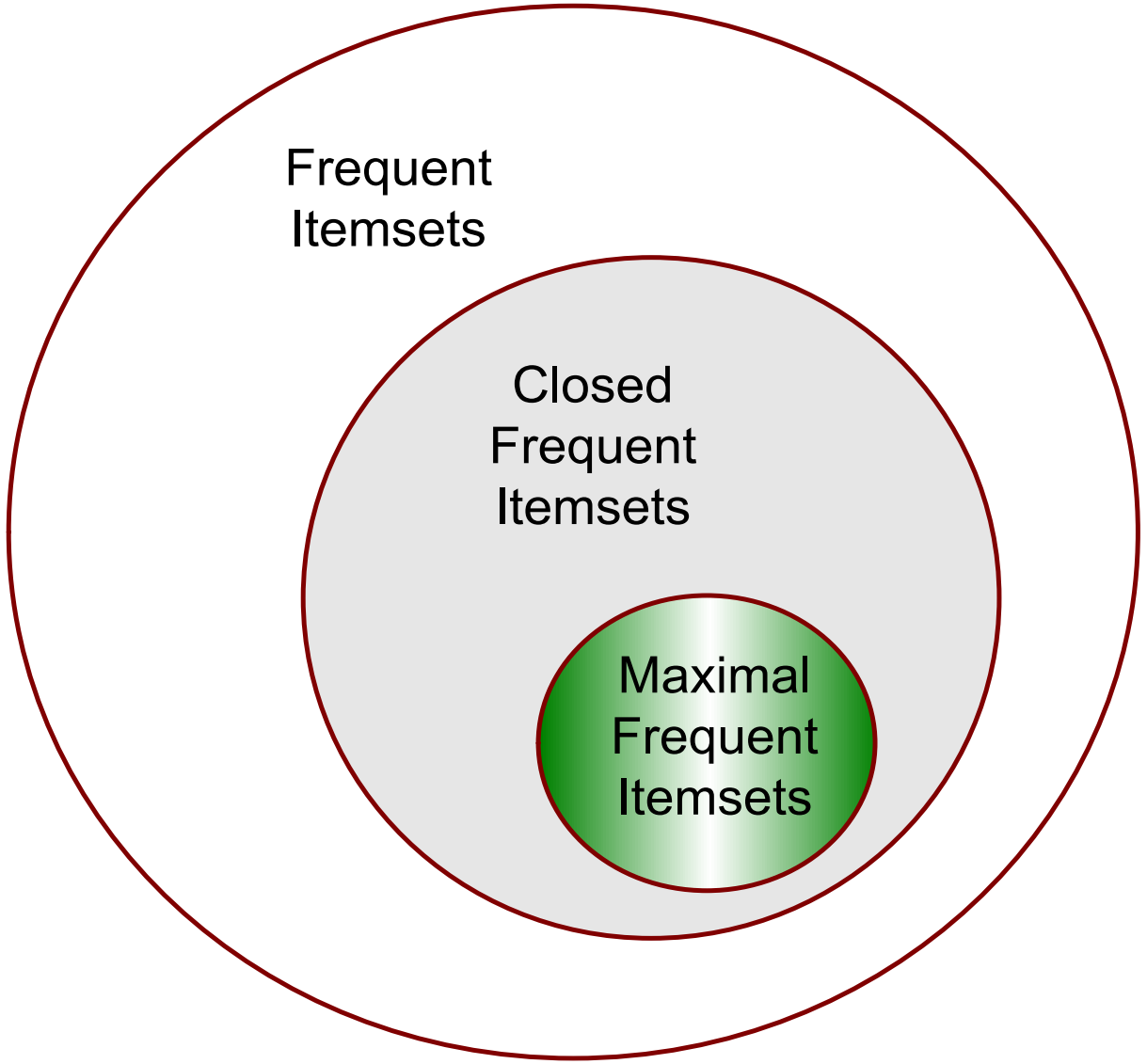

Transaction Ids
null
A
B
C
D
E
124
123
1234
245
345
AB
AC
AD
AE
BC
BD
BE
CD
CE
DE
12
124
24
4
123
2
3
24
34
45
ABC
ABD
ABE
ACD
ACE
ADE
BCD
BCE
BDE
CDE
12
2
24
4
4
2
3
4
ABCD
ABCE
ABDE
ACDE
BCDE
2
4
ABCDE
Not supported by
any transactions



Closed frequent = 9

Maximal frequent = 4

Relationship between frequent itemsets



So far, we know that

- We use support to identify frequent itemsets.
  - Measures how often an itemset occurs in the database.
- Frequent itemsets can be summarized as:
  - Closed: no proper superset has the same support.
  - Maximal: no proper superset is frequent.
- We can use confidence to filter strong association rules.
  - Measures how often the consequent Y occurs when the antecedent X occurs.
- Next question: How can we efficiently find all frequent itemsets?

Apriori Algorithm

1994, VLDB

Fast Algorithms for Mining Association Rules

Rakesh Agrawal Ramakrishnan Srikant*

IBM Almaden Research Center
650 Harry Road, San Jose, CA 95120

Abstract

We consider the problem of discovering association rules between items in a large database of sales transactions. We present two new algorithms for solving this problem that are fundamentally different from the known algorithms. Empirical evaluation shows that these algorithms outperform the known algorithms by factors ranging from three for small problems to more than an order of magnitude for large problems. We also show how the best features of the two proposed algorithms can be combined into a hybrid algorithm, called AprioriHybrid. Scale-up experiments show that AprioriHybrid scales linearly with the number of transactions. AprioriHybrid also has excellent scale-up properties with respect to the transaction size and the number of items in the database.

1 Introduction

Progress in bar-code technology has made it possible for retail organizations to collect and store mas-

tires and auto accessories also get automotive services done. Finding all such rules is valuable for cross-marketing and attached mailing applications. Other applications include catalog design, add-on sales, store layout, and customer segmentation based on buying patterns. The databases involved in these applications are very large. It is imperative, therefore, to have fast algorithms for this task.

The following is a formal statement of the problem [4]: Let $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ be a set of literals, called items. Let $\mathcal{D}$ be a set of transactions, where each transaction T is a set of items such that $T \subseteq \mathcal{I}$. Associated with each transaction is a unique identifier, called its *TID*. We say that a transaction T *contains* X, a set of some items in $\mathcal{I}$, if $X \subseteq T$. An *association rule* is an implication of the form $X \Longrightarrow Y$, where $X \subset \mathcal{I}$, $Y \subset \mathcal{I}$, and $X \cap Y = \emptyset$. The rule $X \Longrightarrow Y$ holds in the transaction set $\mathcal{D}$ with *confidence* c if c% of transactions in $\mathcal{D}$ that contain

Introduction to Apriori algorithm

- If an itemset is frequent, all of its non-empty subsets must also be frequent.
  - $\{b, c, d\}$ is frequent, all of its non-empty subsets
    - $\{b\}, \{c\}, \{d\}, \{b, c\}, \{b, d\}, \{c, d\}$ are frequent
- If an itemset is infrequent, all of its supersets must be infrequent.

The key property of frequent itemsets

- The prior knowledge of frequent itemset properties
  - All non-empty subsets of a frequent itemset must also be frequent
    - If {beer, diaper, nuts} is frequent, so is {beer, diaper}
    - Every transaction containing {beer, diaper, nuts} also contains {beer, diaper}
  - All the supersets of an infrequent itemset must also be infrequent

The Apriori algorithm

- Apriori: Any non-empty subset of a frequent itemset must also be frequent.
  - Finding frequent itemsets through constrained candidate generation (JOIN)
  - If any subset of a candidate itemset S is infrequent, S cannot be frequent and can be pruned before support counting. (PRUNE)
    - Why do we even have to consider S?

Introduction to Apriori algorithm

- If an itemset X is infrequent, none of its supersets can be frequent.

$$sup(X) < \min_sup$$

- For any item $A \notin X$,

$$sup(X \cup \{A\}) \leq sup(X) < \min_sup$$

  - X is a 2-itemset, $X = \{a, b\}, \sup(X) < \min_sup$ → If we add a item c to X, we obtain $\{a, b, c\}$, a 3-itemset.
- Example:
  - If $\{a, b\}$ is infrequent, then $\{a, b, c\}$ must also be infrequent.
  - Therefore, $\{a, b, c\}$ can be pruned before support counting.
    - Check all 2-item subsets of $\{a, b, c\}$: {a, b}, {a, c}, {b, c}

Introduction to Apriori algorithm

- Apriori finds frequent itemsets in multiple passes over the database.
  1. Scan the database to find frequent 1-itemsets, L_1.
  2. At iteration k:
     a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
     b. Prune candidates having an infrequent k-subset.
     c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
  3. Stop when $L_{k+1} = \emptyset$.

Introduction to Apriori algorithm

- Level-wise approach (two-steps, join and prune)
  - Frequent k-itemsets are used to explore $(k + 1)$-itemsets
  - 1-itemsets → 2-itemtsets → 3-itemsets → …
- At each iteration k:
  ① Join step: Combine frequent k-itemsets to form candidate $(k + 1)$-itemsets:

$$C_{k+1} = L_k \bowtie L_k$$

  ② Prune step: Remove any candidate in C_{k+1} if one of its k-subsets is not in L_k, before counting its support.

Apriori: Join and Prune

Apriori algorithm: Join step

- Assume that items within a transaction or itemset are sorted in lexicographic order
  - Let $l_i, l_j \in L_k$, where items are in lexicographic order.
  - Join them only if their first $k-1$ items are identical:
    - If $l_i[1] = l_j[1],\ l_i[2] = l_j[2], \ldots,\ l_i[\mathrm{k}-1] = l_j[k-1],\ l_i[k] < l_j[k]$ is true, we can join l_i, l_j to be the result candidate $\{l_i\,[1], l_i\,[2], \ldots, l_i\,[k], l_j\,[k]\}$
  - Example:
    - $\{a, b, c, d\} \bowtie \{a, b, c, e\} \Rightarrow \{a, b, c, d, e\}$ (we have $\{a, b, c\}$ the same and $d < e$)
    - $\{c, e, f\} \bowtie \{c, e, z\} \Rightarrow \{c, e, f, z\}$
    - $\{a, c, e\} \not\bowtie \{b, c, e\}$: cannot be joined (their first $k-1$ items differ)
    - $\{b, c, e\} \not\bowtie \{b, c, d\}$: cannot be joined ($e > d$)

Why lexicographic order? To ensure that the same candidate set is generated only once.

Apriori algorithm: Prune step

- In the join step, we obtain C_{k+1} from L_k
  - Frequent $(k+1)$-itemsets (L_{k+1}) must come from candidate C_{k+1}

$$L_{k+1} \subseteq C_{k+1}$$

  - Of course, we can directly count the support of every candidate requires a database scan and can be expensive when the candidate set is large.
  - All non-empty subsets of a frequent itemset must also be frequent
  - If any k-subset of a candidate $c \in C_{k+1}$ is not in L_k, then c cannot be frequent and is removed from C_{k+1}
    - For $k = 2$, if we have a candidate $\{a, b, c\} \in C_{k+1}$
    - All 2-subset: $\{a, b\}, \{a, c\}, \{b, c\}$ should belong to L_2. If any one is absent, prune $\{a, b, c\}$.

min_sup count = 2

TID	ITEM
001	a, b, e
002	b, d
003	b, c
004	a, b, d
005	a, c
006	b, c
007	a, c, f
008	a, b, c, e
009	a, b, c

Scan database →

C_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
$\{f\}$	1

count >= 2 →

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
~~$\{f\}$~~	~~1~~

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

k=1 join

C_2

Itemset
$\{a, b\}$
$\{a, c\}$
$\{a, d\}$
$\{a, e\}$
$\{b, c\}$
$\{b, d\}$
$\{b, e\}$
$\{c, d\}$
$\{c, e\}$
$\{d, e\}$

k=1 prune

C_2

Itemset
$\{a, b\}$
$\{a, c\}$
$\{a, d\}$
$\{a, e\}$
$\{b, c\}$
$\{b, d\}$
$\{b, e\}$
$\{c, d\}$
$\{c, e\}$
$\{d, e\}$

count >= 2

L_2

Itemset	count
$\{a, b\}$	4
$\{a, c\}$	4
~~$\{a, d\}$~~	~~1~~
$\{a, e\}$	2
$\{b, c\}$	4
$\{b, d\}$	2
$\{b, e\}$	2
~~$\{c, d\}$~~	~~0~~
~~$\{c, e\}$~~	~~1~~
~~$\{d, e\}$~~	~~0~~

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

$\{c,e\},\{c,d\},\{d,e\}$ are not in L_2

L_2

Itemset	count
$\{a,b\}$	4
$\{a,c\}$	4
$\{a,e\}$	2
$\{b,c\}$	4
$\{b,d\}$	2
$\{b,e\}$	2

k=2 join

C_3

Itemset
$\{a,b,c\}$
$\{a,b,e\}$
$\{a,c,e\}$
$\{b,c,d\}$
$\{b,c,e\}$
$\{b,d,e\}$

k=2 prune

C_3

Itemset
$\{a,b,c\}$
$\{a,b,e\}$
~~$\{a,c,e\}$~~
~~$\{b,c,d\}$~~
~~$\{b,c,e\}$~~
~~$\{b,d,e\}$~~

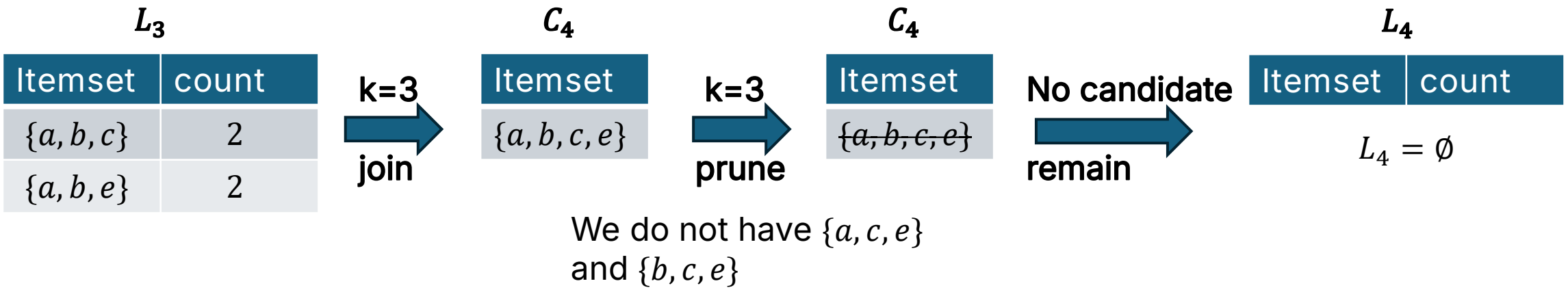
count >= 2

L_3

Itemset	count
$\{a,b,c\}$	2
$\{a,b,e\}$	2

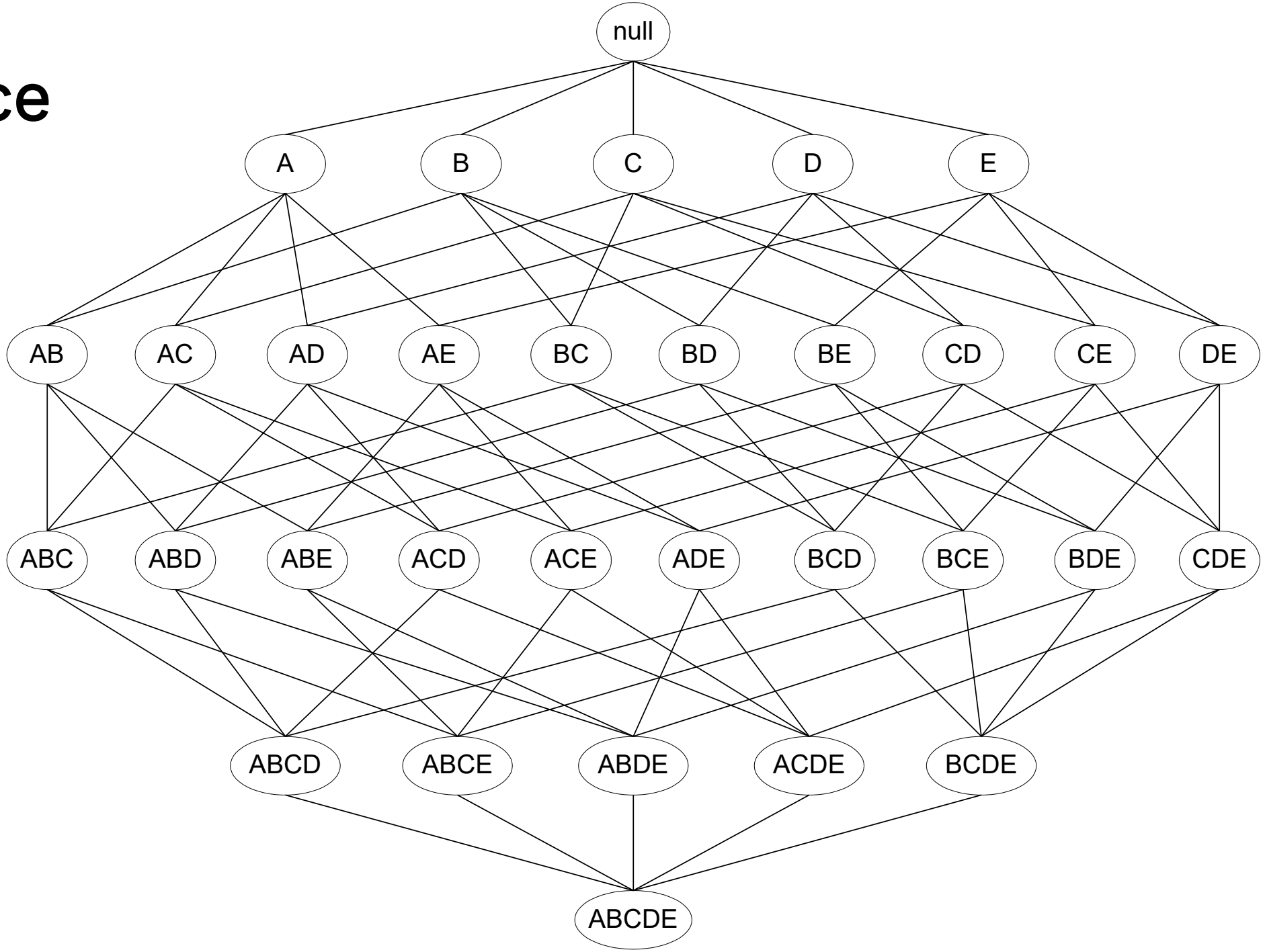
- $\{a,b,c\} \rightarrow \{a,b\},\{a,c\},\{b,c\}$
- $\{a,b,e\} \rightarrow \{a,b\},\{a,e\},\{b,e\}$
- $\{a,c,e\} \rightarrow \{a,c\},\{a,e\},\{c,e\}$
- $\{b,c,d\} \rightarrow \{b,c\},\{b,d\},\{c,d\}$
- $\{b,c,e\} \rightarrow \{b,c\},\{b,e\},\{c,e\}$
- $\{b,d,e\} \rightarrow \{b,d\},\{b,e\},\{d,e\}$

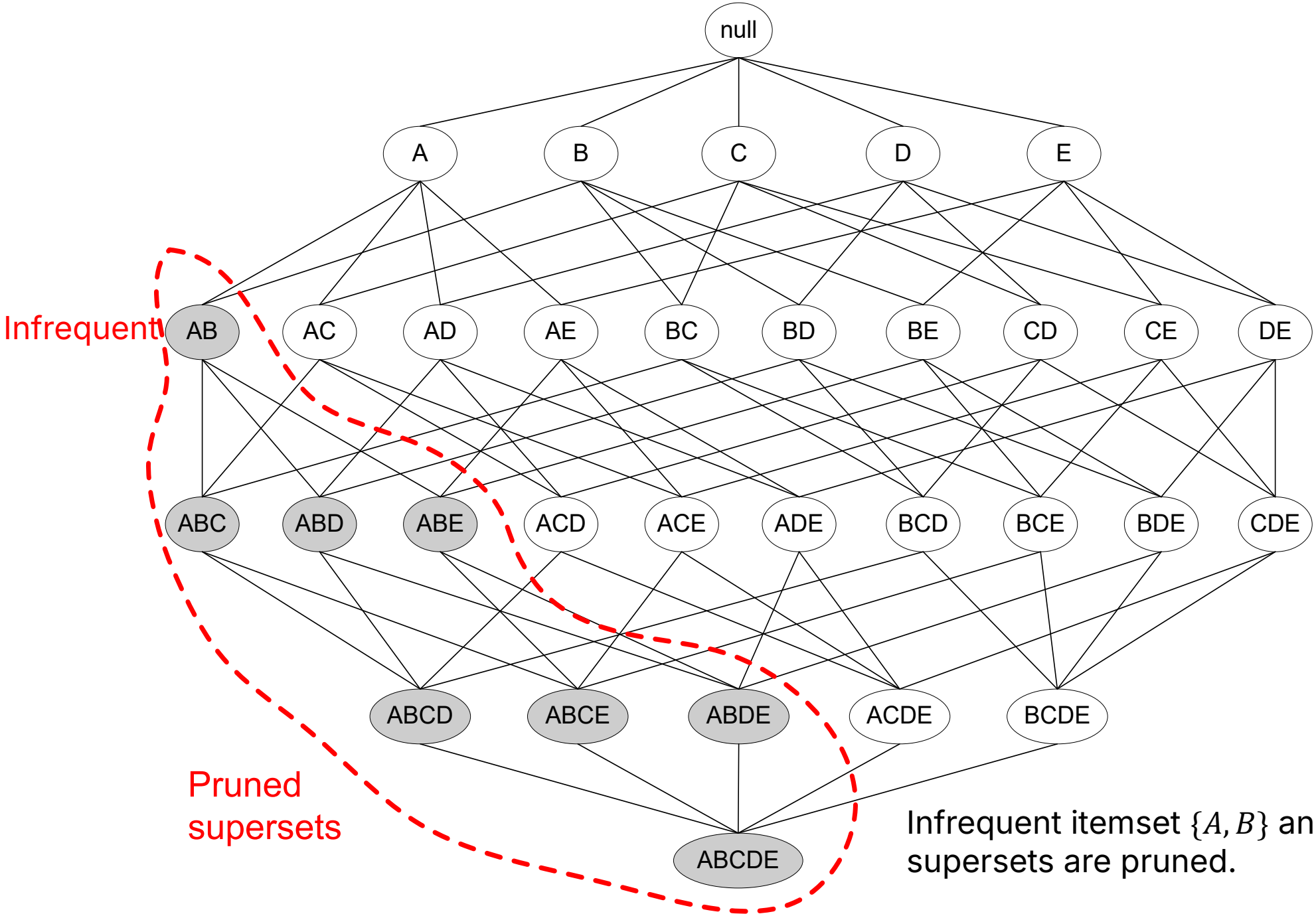
1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.



1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

Lattice


null
A
B
C
D
E
AB
AC
AD
AE
BC
BD
BE
CD
CE
DE
ABC
ABD
ABE
ACD
ACE
ADE
BCD
BCE
BDE
CDE
ABCD
ABCE
ABDE
ACDE
BCDE
ABCDE



Infrequent itemset $\{A, B\}$ and all of its supersets are pruned.

L

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

L_2

Itemset	count
$\{a, b\}$	4
$\{a, c\}$	4
$\{a, e\}$	2
$\{b, c\}$	4
$\{b, d\}$	2
$\{b, e\}$	2

L_3

Itemset	count
$\{a, b, c\}$	2
$\{a, b, e\}$	2

Apriori Algorithm

(1) L_1 = find_frequent_1-itemsets(D);
(2) **for** ($k = 2$; $L_{k-1} \neq \phi$; k++) {
(3) C_k = apriori_gen(L_{k-1}); Candidate
(4) **for each** transaction $t \in D$ { // scan D for counts
(5) C_t = subset(C_k, t); // get the subsets of t that are candidates
(6) **for each** candidate $c \in C_t$ Support counting
(7) c.count++;
(8) }
(9) $L_k = \{c \in C_k | c.count \geq min_sup\}$
(10) }
(11) **return** $L = \cup_k L_k$;

Candidate Generation

procedure apriori_gen(L_{k-1}:frequent $(k-1)$-itemsets)

(1) **for each** itemset $l_1 \in L_{k-1}$

(2) **for each** itemset $l_2 \in L_{k-1}$

(3) **if** $(l_1[1] = l_2[1]) \wedge (l_1[2] = l_2[2])$ Join

$\wedge \ldots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$ **then** {

(4) $c = l_1 \bowtie l_2$; // join step: generate candidates

(5) **if** has_infrequent_subset(c, L_{k-1}) **then** Prune

(6) **delete** c; // prune step: remove unfruitful candidate

(7) **else add** c **to** C_k;

(8) }

(9) **return** C_k;

Candidate Generation

```
procedure has_infrequent_subset(c: candidate k-itemset;
                L_{k-1}: frequent (k − 1)-itemsets); // use prior knowledge
(1)     for each (k − 1)-subset s of c
(2)         if s ∉ L_{k-1} then
(3)             return TRUE;
(4)     return FALSE;
```

Don’t forget !

$$confidence(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

- Apriori only finds frequent itemsets
- Association rules are generated afterward
- From Frequent Itemsets to Association Rules
  1. Use Apriori to get all frequent itemsets (L)
  2. Generate association rules from those itemsets
     - For each frequent itemset $l \in L$ with $|l| \geq 2$, enumerate every nonempty proper subset $s \subset l$, and generate

$$s \rightarrow (l - s)$$

if

$$\frac{support_count\ (l)}{support_count\ (s)} \geq \min_conf$$

Finding association rules using Apriori

- Generate association rules from those itemsets
  - Suppose we find a frequent itemset X = $\{a, b, e\}$ using Apriori
  - Subset of X is: $\{a, b\}, \{a, e\}, \{b, e\}, \{a\}, \{b\}, \{e\}$

1. $\{a, b\} \rightarrow \{e\}$ (e.g., $\{a, b, e\} - \{a, b\} = \{e\}$): $sup(\{a, b, e\}) \,/\, sup\,\{a, b\}$
2. $\{a, e\} \rightarrow \{b\}$: $sup(\{a, b, e\}) \,/\, sup(\{a, e\})$
3. $\{b, e\} \rightarrow \{a\}$: $sup(\{a, b, e\}) \,/\, sup(\{b, e\})$
4. $\{a\} \rightarrow \{b, e\}$: $sup(\{a, b, e\}) \,/\, sup(\{a\})$
5. $\{b\} \rightarrow \{a, e\}$: $sup(\{a, b, e\}) \,/\, sup(\{b\})$
6. $\{e\} \rightarrow \{a, b\}$: $sup(\{a, b, e\}) \,/\, sup(\{e\})$

- Frequent itemsets and their support counts can be stored in a hash table for efficient lookup.

min_sup count = 2

TID	ITEM
001	a, b, e
002	b, d
003	b, c
004	a, b, d
005	a, c
006	b, c
007	a, c, f
008	a, b, c, e
009	a, b, c

Scan database

C_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
$\{f\}$	1

count >= 2

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
~~$\{f\}$~~	~~1~~

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

k=1 join

C_2

Itemset
$\{a,b\}$
$\{a,c\}$
$\{a,d\}$
$\{a,e\}$
$\{b,c\}$
$\{b,d\}$
$\{b,e\}$
$\{c,d\}$
$\{c,e\}$
$\{d,e\}$

k=1 prune

C_2

Itemset
$\{a,b\}$
$\{a,c\}$
$\{a,d\}$
$\{a,e\}$
$\{b,c\}$
$\{b,d\}$
$\{b,e\}$
$\{c,d\}$
$\{c,e\}$
$\{d,e\}$

count >= 2

L_2

Itemset	count
$\{a,b\}$	4
$\{a,c\}$	4
~~$\{a,d\}$~~	~~1~~
$\{a,e\}$	2
$\{b,c\}$	4
$\{b,d\}$	2
$\{b,e\}$	2
~~$\{c,d\}$~~	~~0~~
~~$\{c,e\}$~~	~~1~~
~~$\{d,e\}$~~	~~0~~

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

$\{c,e\},\{c,d\},\{d,e\}$ are not in L_2

L_2

Itemset	count
$\{a,b\}$	4
$\{a,c\}$	4
$\{a,e\}$	2
$\{b,c\}$	4
$\{b,d\}$	2
$\{b,e\}$	2

k=2 join

C_3

Itemset
$\{a,b,c\}$
$\{a,b,e\}$
$\{a,c,e\}$
$\{b,c,d\}$
$\{b,c,e\}$
$\{b,d,e\}$

k=2 prune

C_3

Itemset
$\{a,b,c\}$
$\{a,b,e\}$
~~$\{a,c,e\}$~~
~~$\{b,c,d\}$~~
~~$\{b,c,e\}$~~
~~$\{b,d,e\}$~~

count >= 2

L_3

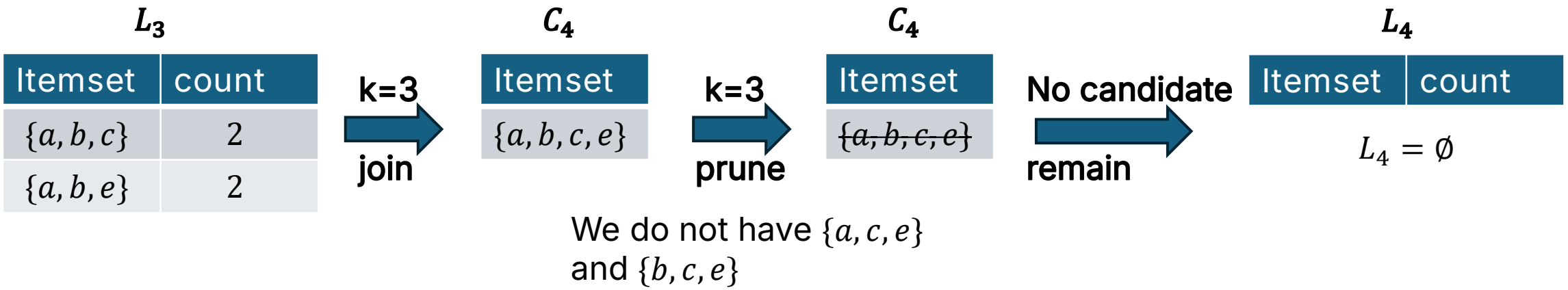
Itemset	count
$\{a,b,c\}$	2
$\{a,b,e\}$	2

- $\{a,b,c\} \rightarrow \{a,b\},\{a,c\},\{b,c\}$
- $\{a,b,e\} \rightarrow \{a,b\},\{a,e\},\{b,e\}$
- $\{a,c,e\} \rightarrow \{a,c\},\{a,e\},\{c,e\}$
- $\{b,c,d\} \rightarrow \{b,c\},\{b,d\},\{c,d\}$

  $\{b,c,e\} \rightarrow \{b,c\},\{b,e\},\{c,e\}$

  $\{b,d,e\} \rightarrow \{b,d\},\{b,e\},\{d,e\}$

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.



1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

L

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

L_2

Itemset	count
$\{a,b\}$	4
$\{a,c\}$	4
$\{a,e\}$	2
$\{b,c\}$	4
$\{b,d\}$	2
$\{b,e\}$	2

L_3

Itemset	count
$\{a,b,c\}$	2
$\{a,b,e\}$	2

L_2 association rules:

$\{a,b\}$: subset $\{a\}$ and $\{b\}$ ➔ $\{a\} \rightarrow \{b\} \left(\frac{count(\{a,b\})}{count(\{a\})} = \frac{4}{6}\right), \{b\} \rightarrow \{a\} \left(\frac{4}{7}\right)$

$\{a,c\}$: subset $\{a\}$ and $\{c\}$ ➔ $\{a\} \rightarrow \{c\} \left(\frac{4}{6}\right), \{c\} \rightarrow \{a\} \left(\frac{4}{6}\right)$

$\{a,e\}$: subset $\{a\}$ and $\{e\}$ ➔ $\{a\} \rightarrow \{e\} \left(\frac{2}{6}\right), \{e\} \rightarrow \{a\} \left(\frac{2}{2}\right)$

$\{b,c\}$: subset $\{b\}$ and $\{c\}$ ➔ $\{b\} \rightarrow \{c\} \left(\frac{4}{7}\right), \{c\} \rightarrow \{b\} \left(\frac{4}{6}\right)$

$\{b,d\}$: subset $\{b\}$ and $\{d\}$ ➔ $\{b\} \rightarrow \{d\} \left(\frac{2}{7}\right), \{d\} \rightarrow \{b\} \left(\frac{2}{2}\right)$

$\{b,e\}$: subset $\{b\}$ and $\{e\}$ ➔ $\{b\} \rightarrow \{e\} \left(\frac{2}{7}\right), \{e\} \rightarrow \{b\} \left(\frac{2}{2}\right)$

L

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

L_2

Itemset	count
$\{a, b\}$	4
$\{a, c\}$	4
$\{a, e\}$	2
$\{b, c\}$	4
$\{b, d\}$	2
$\{b, e\}$	2

L_3

Itemset	count
$\{a, b, c\}$	2
$\{a, b, e\}$	2

L_3 association rules:

$\{a, b, c\}$: subset $\{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}$

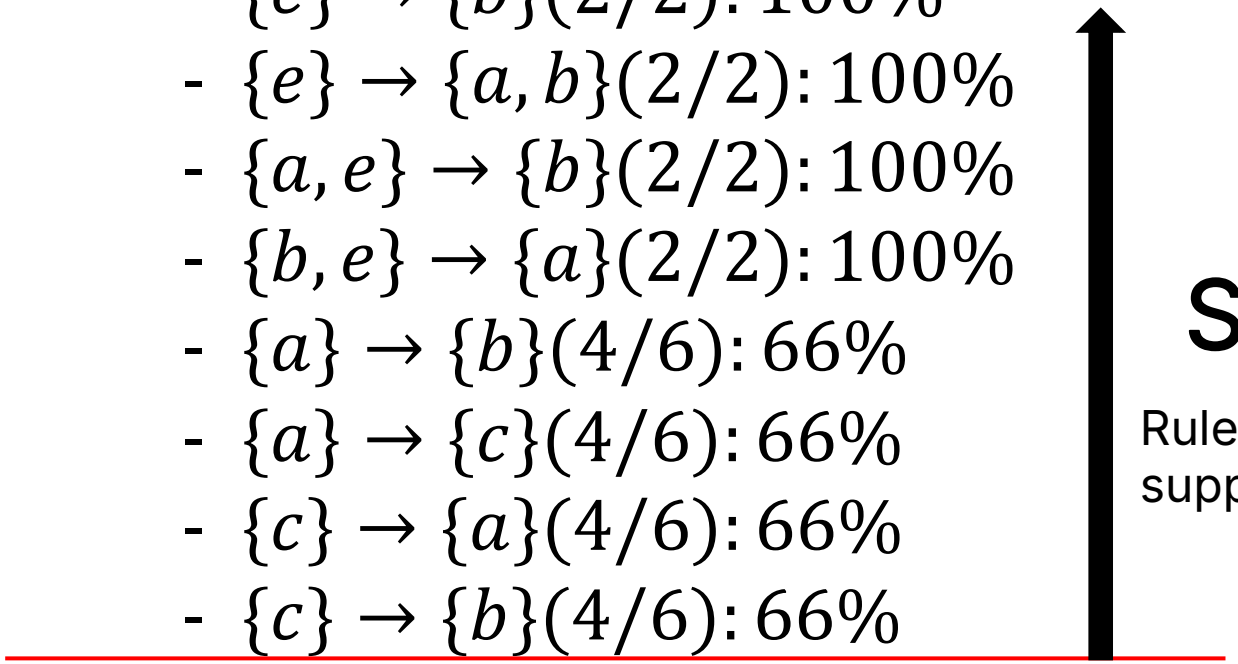
$$\{a\} \rightarrow \{b, c\}\left(\frac{2}{6}\right), \{\mathrm{b}\} \rightarrow \{\mathrm{a}, \mathrm{c}\}\left(\frac{2}{7}\right), \{\mathrm{c}\} \rightarrow \{\mathrm{a}, \mathrm{b}\}\left(\frac{2}{6}\right), \{\mathrm{a}, \mathrm{b}\} \rightarrow \{\mathrm{c}\}\left(\frac{2}{4}\right), \{\mathrm{a}, \mathrm{c}\} \rightarrow \{b\}\left(\frac{2}{4}\right), \{b, c\} \rightarrow \{a\}\left(\frac{2}{4}\right)$$

$\{a, b, e\}$: subset $\{a\}, \{b\}, \{e\}, \{a, b\}, \{a, e\}, \{b, e\}$

$$\{a\} \rightarrow \{b, e\}\left(\frac{2}{6}\right), \{\mathrm{b}\} \rightarrow \{\mathrm{a}, \mathrm{e}\}\left(\frac{2}{7}\right), \{\mathrm{e}\} \rightarrow \{\mathrm{a}, \mathrm{b}\}\left(\frac{2}{2}\right), \{\mathrm{a}, \mathrm{b}\} \rightarrow \{\mathrm{e}\}\left(\frac{2}{4}\right), \{\mathrm{a}, \mathrm{e}\} \rightarrow \{b\}\left(\frac{2}{2}\right), \{b, e\} \rightarrow \{a\}\left(\frac{2}{2}\right)$$

Minimum confidence: 60%

- $\{e\} \rightarrow \{a\}(2/2): 100\%$
- $\{d\} \rightarrow \{b\}(2/2): 100\%$
- $\{e\} \rightarrow \{b\}(2/2): 100\%$
- $\{e\} \rightarrow \{a, b\}(2/2): 100\%$
- $\{a, e\} \rightarrow \{b\}(2/2): 100\%$
- $\{b, e\} \rightarrow \{a\}(2/2): 100\%$
- $\{a\} \rightarrow \{b\}(4/6): 66\%$
- $\{a\} \rightarrow \{c\}(4/6): 66\%$
- $\{c\} \rightarrow \{a\}(4/6): 66\%$
- $\{c\} \rightarrow \{b\}(4/6): 66\%$



- $\{b\} \rightarrow \{a\}(4/7): 57\%$
- $\{b\} \rightarrow \{c\}(4/7): 57\%$
- …
- …
- …

Strong Rules

Rules satisfying both minimum support and minimum confidence.

Next Week: Improving the efficiency of Apriori

- Apriori optimizations
  - Hash-based technique
  - Transaction reduction
  - Partitioning
  - Sampling
- Beyond Apriori
  - FP-Tree / FP-Growth